

Lecture 2:

Linear Algebra Fundamentals



Markus Hohle

University California, Berkeley

**Numerical Methods for
Computational Science**

MSSE 273, 3 Units

Course Map

Week 1: Introduction to Scientific Computing and Python Libraries

Week 2: Linear Algebra Fundamentals

Week 3: Vector Calculus

Week 4: Numerical Differentiation and Integration

Week 5: Solving Nonlinear Equations

Week 6: Probability Theory Basics

Week 7: Random Variables and Distributions

Week 8: Statistics for Data Science

Week 9: Eigenvalues and Eigenvectors

Week 10: Simulation and Monte Carlo Method

Week 11: Data Fitting and Regression

Week 12: Optimization Techniques

Week 13: Machine Learning Fundamentals



Outline

- Arrays in Python
- Vectors
- Matrices
- Lecture Exercise
- Solving Linear Systems



Outline

- Arrays in Python

- Vectors

- Matrices

- Lecture Exercise

- Solving Linear Systems



shapes:

```
A = np.arange(0,5,1)
```

```
In [36]: print(A)  
[0 1 2 3 4]
```

```
In [37]: A.shape  
Out[37]: (5,)
```

```
A = range(0,5,1)
```

```
In [40]: print(A)  
range(0, 5)
```

```
In [41]: A.shape  
Traceback (most recent call last):  
  
  Cell In[41], line 1  
    A.shape  
AttributeError: 'range' object has no attribute 'shape'
```

```
A = np.zeros((1,5))
```

```
In [53]: print(A)  
[[0. 0. 0. 0. 0.]]
```

```
In [54]: A.shape  
Out[54]: (1, 5)
```

Make sure that shapes make sense!

```
In [60]: A = np.arange(0,5,1)
```

```
In [61]: Anew = A.reshape((1,5))
```

```
In [62]: Anew.shape  
Out[62]: (1, 5)
```



shapes:

```
A = np.arange(0,5,1)
```

```
In [36]: print(A)  
[0 1 2 3 4]
```

```
In [65]: print(type(A))  
<class 'numpy.ndarray'>
```

```
A = range(0,5,1)
```

```
In [71]: print(type(A))  
<class 'range'>
```

```
A = np.zeros((1,5))
```

```
In [53]: print(A)  
[[0. 0. 0. 0. 0.]]
```

```
In [68]: print(type(A))  
<class 'numpy.ndarray'>
```




Outline

- Arrays in Python

- **Vectors**

- Matrices

- Lecture Exercise

- Solving Linear Systems



recap:

quantities with **magnitude** and **direction**

→ vectors $\vec{v}$

- velocity
- current
- acceleration
- force
- electric and magnetic field
- ...

quantities with **magnitude**, **no direction**

→ scalars S

- density (and all related quantities
like concentration, pressure etc)
- mass
- energy
- ...



Why do we need vectors:

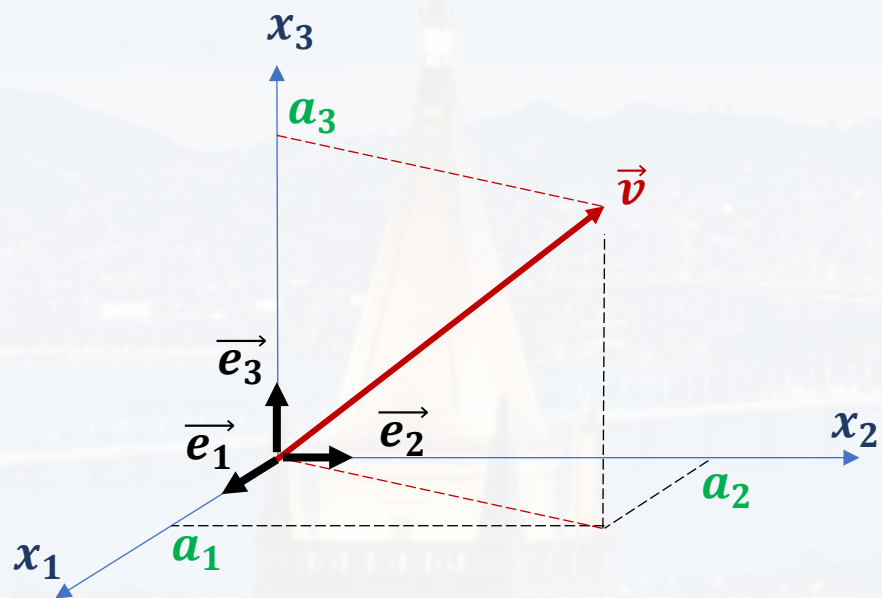
- “vectorized” code is :
 - faster
 - shorter
 - maintainable
 - readable
 - scalable

→ see lecture exercise
- data analysis and data acquisition of physical problems requires basic understanding
- many ML algorithms and tools are inspired by physics
- dimension reduction, PCA (eigenvectors, eigenvalues)



recap:

Cartesian coordinates



unit vectors $\vec{e}_1$, $\vec{e}_2$ and $\vec{e}_3$

$$\vec{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \vec{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \vec{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

length 1 (normalized)
mutually orthogonal } ortho normal

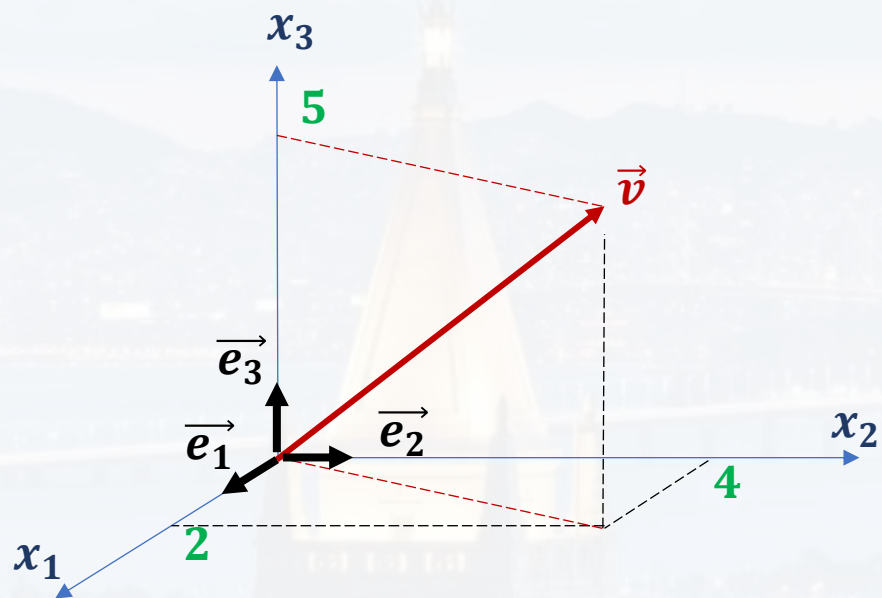
$$\vec{v} = a_1 \vec{e}_1 + a_2 \vec{e}_2 + a_3 \vec{e}_3$$

$$\vec{v} = a_1 \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + a_2 \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + a_3 \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix}$$



recap:

Cartesian coordinates



unit vectors $\vec{e}_1$, $\vec{e}_2$ and $\vec{e}_3$

$$\vec{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \vec{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \vec{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

length 1 (normalized)
mutually orthogonal } ortho normal

$$\vec{v} = a_1 \vec{e}_1 + a_2 \vec{e}_2 + a_3 \vec{e}_3$$

$$\vec{v} = 2 \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + 4 \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} + 5 \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 4 \\ 5 \end{pmatrix}$$



recap:

FYI:

- non orthogonal coordinate systems (e. g. crystal lattices)
→ covariant and contravariant vectors
- curved ortho normal coordinates (polar coordinates, spherical coordinates etc.)
- general coordinates: curved, not normalized and not orthogonal (**not important for this course**)



recap:

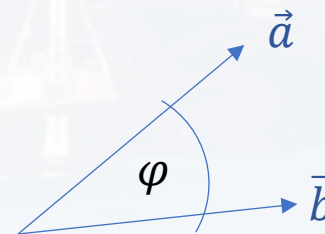
dot product of two vectors

$$\vec{a} \circ \vec{b} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \circ \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

$$\vec{a} \circ \vec{b} = \sum_{i=1}^I a_i b_i$$

$$\cos(\varphi) = \frac{\vec{a} \circ \vec{b}}{|\vec{a}| |\vec{b}|}$$

angle between $\vec{a}$ and $\vec{b}$



$|\vec{a}|$ length of $\vec{a}$



recap:

dot product of two vectors

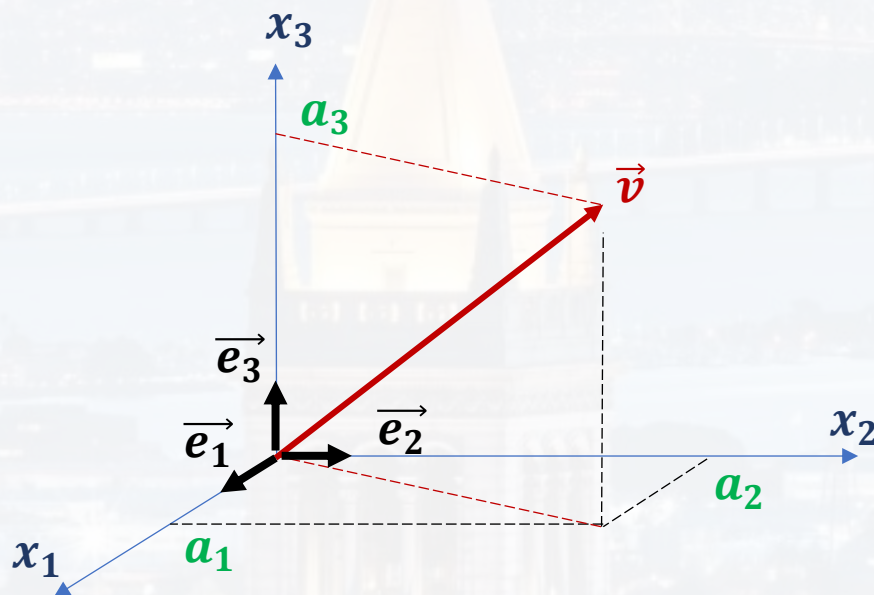
$$\vec{a} \circ \vec{b} = \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} \circ \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

$$\vec{a} \circ \vec{b} = \sum_{i=1}^I a_i b_i$$

“physical length” L of a vector

$$L = |\vec{a}| = \sqrt{a_1^2 + a_2^2 + a_3^2}$$

$$= \sqrt{\vec{a} \circ \vec{a}}$$





recap:

dot product of two vectors

$$\cos(\varphi) = \frac{\vec{a} \circ \vec{b}}{|\vec{a}| |\vec{b}|}$$

angle between $\vec{a}$ and $\vec{b}$

$$\vec{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \vec{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \vec{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

$$\vec{e}_i \circ \vec{e}_j = \begin{cases} \text{zero for } i \neq j \\ 1 \text{ for } i = j \end{cases}$$
$$\vec{e}_i \circ \vec{e}_j = \delta_{ij} \quad \text{Kronecker symbol } \delta_{ij} = \begin{cases} 0 \text{ for } i \neq j \\ 1 \text{ for } i = j \end{cases}$$



Outline

- Arrays in Python
- Vectors
- **Matrices**
- Lecture Exercise
- Solving Linear Systems



Why do we need matrices:

- “vectorized” code is :
 - faster
 - shorter
 - maintainable
 - readable
 - scalable

→ see lecture exercise
- AI/ML: essentially matrix operations
- regression, linear models etc
- easy to parallelize



in python:

```
[[ 3 -1  2  2]
 [15 -5 10 10]
 [ 0  0  0  0]
 [-9  3 -6 -6]]
```

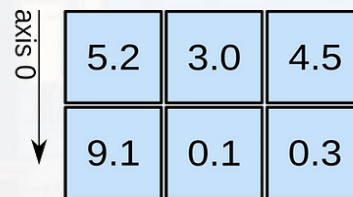
see [here](#)

1D array



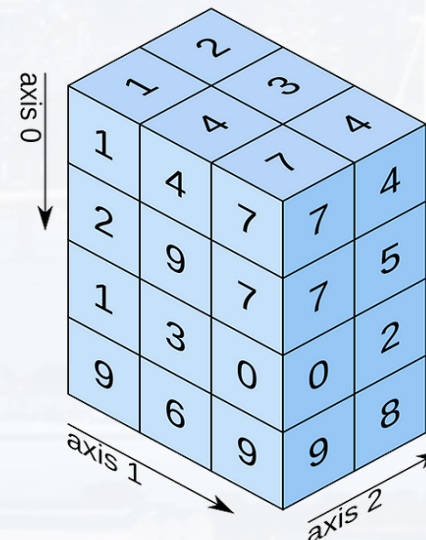
shape: (4,)

2D array



shape: (2, 3)

3D array



shape: (4, 3, 2)

... and higher

note:

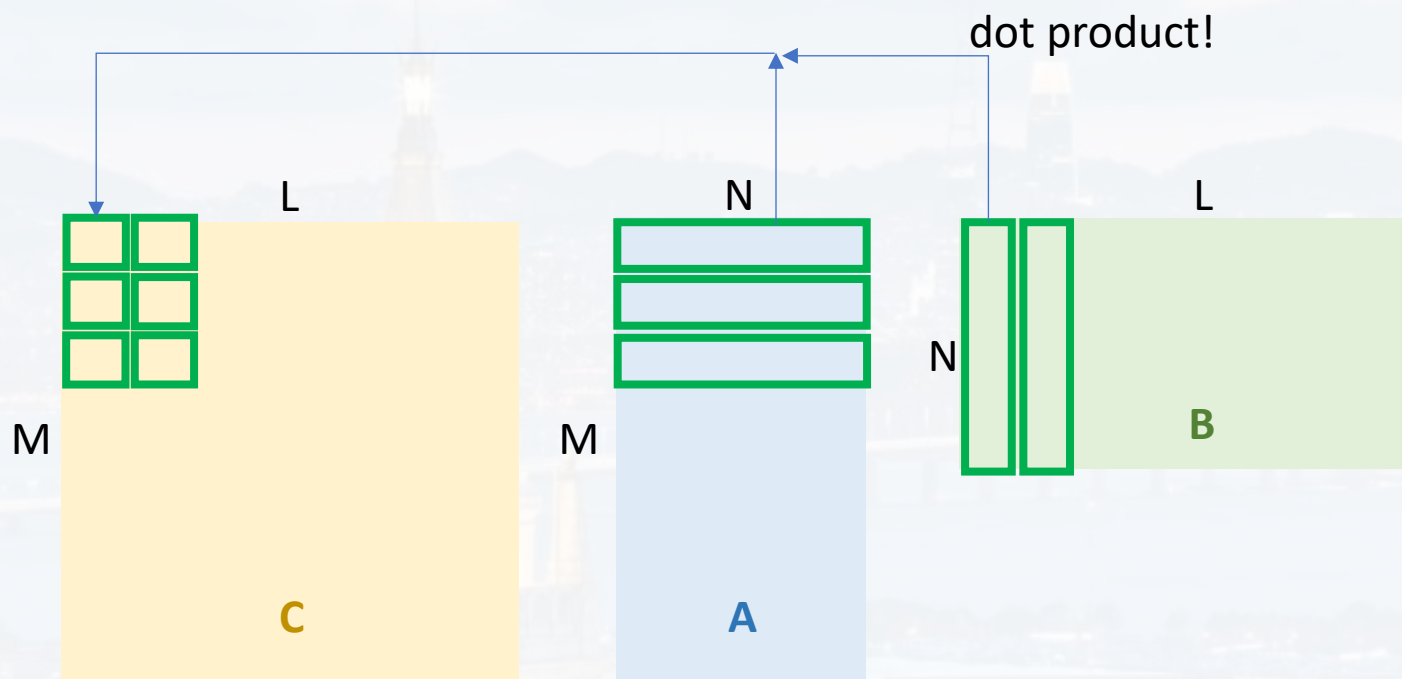
higher dimensional arrays are called **tensor** in the CS community, but they are not the same tensors as in math & physics



in math:

actual matrix multiplication

$$\mathbf{C} = \mathbf{A} * \mathbf{B}$$



$$c_{m,l} = \sum_{n=0}^N a_{m,n} b_{n,l}$$

- only works if $n_{\text{column}}(\mathbf{A}) = n_{\text{row}}(\mathbf{B})$
- result: $\mathbf{C}(n_{\text{row}}(\mathbf{A}), n_{\text{column}}(\mathbf{B}))$



```
v1 = np.array([1, 5, 0, -3])
```

```
v2 = np.array([3, -1, 2, 2])
```

1) `np.dot(v1, v2)`

2) `np.outer(v1, v2)`

$$(a \quad b \quad c) \begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} = a\alpha + b\beta + c\gamma$$

$$\begin{pmatrix} \alpha \\ \beta \\ \gamma \end{pmatrix} (a \quad b \quad c) = \begin{pmatrix} a\alpha & \alpha b & \alpha c \\ a\beta & \beta b & \beta c \\ a\gamma & \gamma b & \gamma c \end{pmatrix}$$

```
In [75]: print(np.dot(v1, v2))  
-8
```

```
In [76]: print(np.dot(v2, v1))  
-8
```

```
In [78]: print(np.outer(v1, v2))  
[[ 3 -1  2  2]  
 [15 -5 10 10]  
 [ 0  0  0  0]  
 [-9  3 -6 -6]]
```

```
In [79]: print(np.outer(v2, v1))  
[[ 3 15  0 -9]  
 [-1 -5  0  3]  
 [ 2 10  0 -6]  
 [ 2 10  0 -6]]
```




```
v1 = np.array([1,5,0,-3])  
v2 = np.array([3,-1,2,2])
```

```
In [81]: print(v1*v2)  
[ 3 -5  0 -6]
```

default: element wise multiplication

```
In [82]: print(v2*v1)  
[ 3 -5  0 -6]
```

```
In [84]: v1v2 = v1*v2
```

```
In [85]: v1v2.shape  
Out[85]: (4,)
```

```
In [86]: v2v1 = v2*v1
```

```
In [87]: v2v1.shape  
Out[87]: (4,)
```



```
v1 = np.array([1,5,0,-3])
```

```
v2 = np.array([3,-1,2,2])
```

```
v1 = v1.reshape((len(v1),1))
```

column vector

```
v2 = v2.reshape((1,len(v2)))
```

row vector

$v1 * v2[0]$

```
In [94]: print(v1*v2)
```

```
[[ 3 -1  2  2]
 [15 -5 10 10]
 [ 0  0  0  0]
 [-9  3 -6 -6]]
```

$v1 * v2[1]$

```
In [95]: print(v2*v1)
```

```
[[ 3 -1  2  2]
 [15 -5 10 10]
 [ 0  0  0  0]
 [-9  3 -6 -6]]
```

note:
operations in Python
are not exactly like those in
math!

more info [here](#)



```
v1 = np.array([1, 5, 0, -3])
```

```
v2 = np.array([3, -1, 2, 2])
```

dot product:

```
np.dot(v1, v1)**0.5
```

“physical length” L of a vector

$$\cos(\varphi) = \frac{\vec{v}_1 \cdot \vec{v}_2}{|\vec{v}_1| |\vec{v}_2|}$$

angle between $v1$ and $v2$

```
np.arccos(np.dot(v1, v2) / np.sum(v1**2)**0.5 / np.sum(v2**2)**0.5)
```




```
v1 = np.array([1, 5, 0, -3])
```

```
v2 = np.array([3, -1, 2, 2])
```

```
np.multiply(v1, v2)
```

```
In [119]: np.multiply(v1, v2)
```

```
Out[119]:
```

```
array([[ 3, -1,  2,  2],  
       [15, -5, 10, 10],  
       [ 0,  0,  0,  0],  
       [-9,  3, -6, -6]])
```

$v2 * v1[0]$

$v2 * v1[1]$

```
In [120]: np.multiply(v2, v1)
```

```
Out[120]:
```

```
array([[ 3, -1,  2,  2],  
       [15, -5, 10, 10],  
       [ 0,  0,  0,  0],  
       [-9,  3, -6, -6]])
```



```
v1 = np.array([1,5,0,-3])  
v2 = np.array([3,-1,2,2])
```

```
v1 = v1.reshape((len(v1),1))  
v2 = v2.reshape((1,len(v2)))
```

```
np.multiply(v1,v2)
```

```
In [119]: np.multiply(v1,v2)
```

```
Out[119]:
```

```
array([[ 3, -1,  2,  2],  
       [15, -5, 10, 10],  
       [ 0,  0,  0,  0],  
       [-9,  3, -6, -6]])
```

```
In [120]: np.multiply(v2,v1)
```

```
Out[120]:
```

```
array([[ 3, -1,  2,  2],  
       [15, -5, 10, 10],  
       [ 0,  0,  0,  0],  
       [-9,  3, -6, -6]])
```

```
In [122]: np.multiply(v1,v2)
```

```
Out[122]: array([ 3, -5,  0, -6])
```

```
In [123]: np.multiply(v2,v1)
```

```
Out[123]: array([ 3, -5,  0, -6])
```



```
array([[ 1,  2,  5,  6],
       [ 0,  8,  5, -4],
       [-4,  4,  6,  1],
       [ 2,  3, -1,  0]])
```

$$M^*M$$
 M^{**2}

```
np.outer(M,M)
```

[illegible]



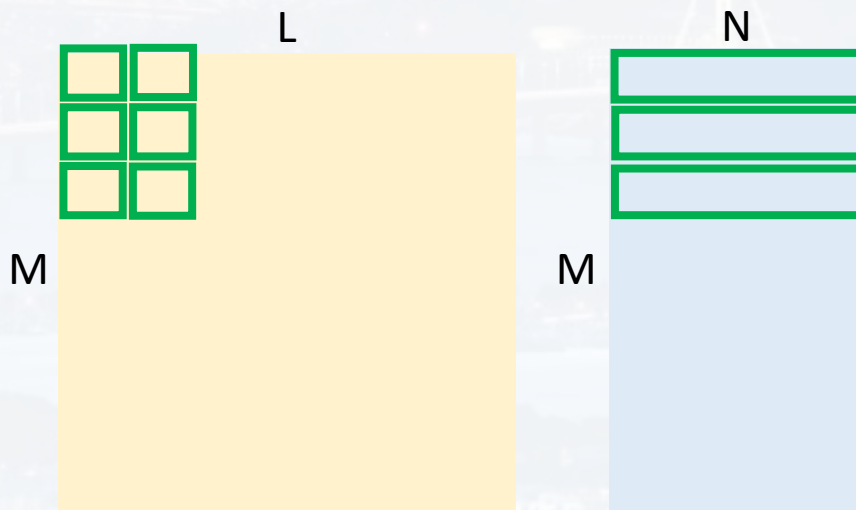
```
M = np.array([[1, 2, 5, 6], [0, 8, 5, -4], [-4, 4, 6, 1],\n              [2, 3, -1, 0]])
```

```
array([[ 1,  2,  5,  6],\n       [ 0,  8,  5, -4],\n       [-4,  4,  6,  1],\n       [ 2,  3, -1,  0]])
```

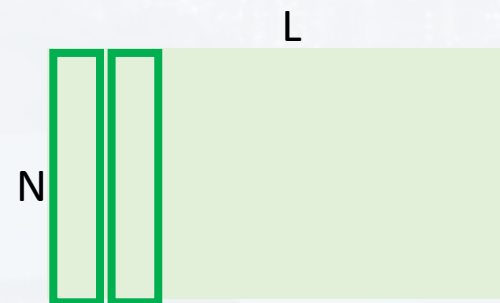
```
np.dot(M,M)
```

```
[[ -7,  56,  39,   3],\n [-28,  72,  74, -27],\n [-26,  51,  35, -34],\n [  6,  24,  19, -1]]
```

actual matrix multiplication



$C = A * B$



$$c_{m,l} = \sum_{n=0}^N a_{m,n} b_{n,l}$$



identity matrix

$$I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$I = \begin{pmatrix} 1 & \dots & 0 \\ \vdots & 1 & \vdots \\ 0 & \dots & 1 \end{pmatrix}$$

identity:

$$I M = M$$

inverse:

$$M^{-1} M = I$$

transpose:

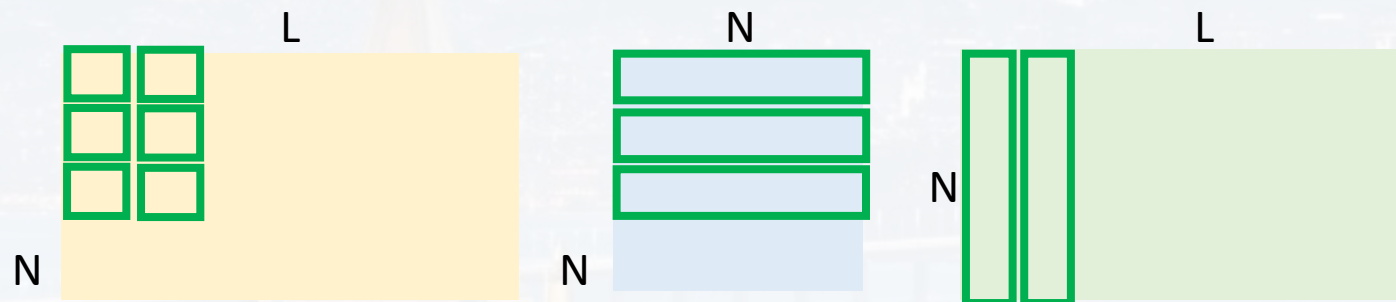
$$[m_{ij}]^T = [m_{ji}]$$

symmetry:

$$[m_{ij}] = [m_{ji}]$$

actual matrix multiplication

$$B = I * B$$



$$c_{m,l} = \sum_{n=0}^N a_{m,n} b_{n,l}$$

`np.eye(5)`

```
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.],
       [0., 0., 0., 1., 0.],
       [0., 0., 0., 0., 1.]])
```



$$MM^{-1} = I$$

```
M_1 = np.linalg.inv(M)
```

```
print(np.dot(M,M_1))
```

identity:	$I M = M$
inverse:	$M^{-1} M = I$
transpose:	$[m_{ij}]^T = [m_{ji}]$
symmetry:	$[m_{ij}] = [m_{ji}]$

```
[ [ 1.00000000e+00  5.55111512e-17  5.55111512e-17 -1.11022302e-16]
  [ 0.00000000e+00  1.00000000e+00  5.55111512e-17 -1.11022302e-16]
  [-4.16333634e-17  2.77555756e-17  1.00000000e+00  5.55111512e-17]
  [ 5.55111512e-17 -5.55111512e-17  8.32667268e-17  1.00000000e+00]]
```

$$I = \begin{pmatrix} 1 & \cdots & 0 \\ \vdots & 1 & \vdots \\ 0 & \cdots & 1 \end{pmatrix}$$

½ machine epsilon
¼ machine epsilon

numerical accuracy
of the operation
(machine epsilon)



```
M = np.array([[1,2,3,4], [5,6,7,8],\
              [9,10,11,12], [13,14,15,16]])
```

identity:	$I M = M$
inverse:	$M^{-1} M = I$
transpose:	$[m_{ij}]^T = [m_{ji}]$
symmetry:	$[m_{ij}] = [m_{ji}]$

```
In [9]: M
Out[9]:
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])
```

```
In [10]: M.transpose()
Out[10]:
array([[ 1,  5,  9, 13],
       [ 2,  6, 10, 14],
       [ 3,  7, 11, 15],
       [ 4,  8, 12, 16]])
```



usually (but not always!) **distance** matrices

- see lecture exercise
- some ML algorithms (trees)

identity:	$I M = M$
inverse:	$M^{-1} M = I$
transpose:	$[m_{ij}]^T = [m_{ji}]$
symmetry:	$[m_{ij}] = [m_{ji}]$

	0	1	2	3	4	5	6	7
0	0	0.118625	0.235409	0.000235242	0.0545817	0.0351447	0.0947077	0.148681
1	0.118625	0	0.0198161	0.129425	0.0122749	0.282906	0.42532	0.00169516
2	0.235409	0.0198161	0	0.250527	0.0632834	0.452469	0.628746	0.00991963
3	0.000235242	0.129425	0.250527	0	0.0619835	0.0296293	0.0855028	0.160744
4	0.0545817	0.0122749	0.0632834	0.0619835	0	0.177322	0.293085	0.0230932
5	0.0351447	0.282906	0.452469	0.0296293	0.177322	0	0.0144665	0.328399
6	0.0947077	0.42532	0.628746	0.0855028	0.293085	0.0144665	0	0.480718
7	0.148681	0.00169516	0.00991963	0.160744	0.0230932	0.328399	0.480718	0



Outline

- Arrays in Python
- Vectors
- Matrices
- **Lecture Exercise**
- Solving Linear Systems



Let's combine some of those commands we just have learned

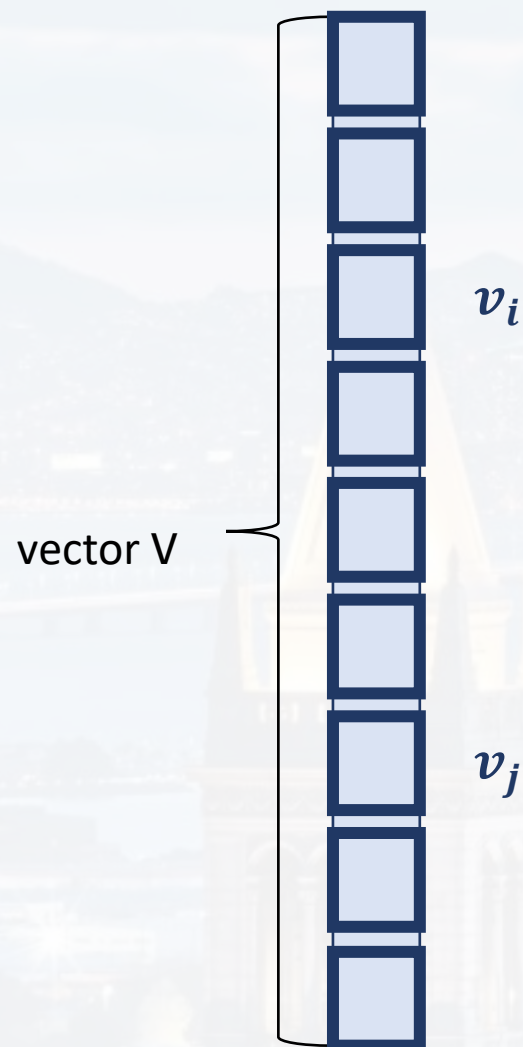
```
np.zeros()  
np.arange()  
np.tile()  
np.transpose()
```

for a particular example about **avoiding loops!**



for loops are simple, but slow → use lin algebra or numpy commands

calculating distances d



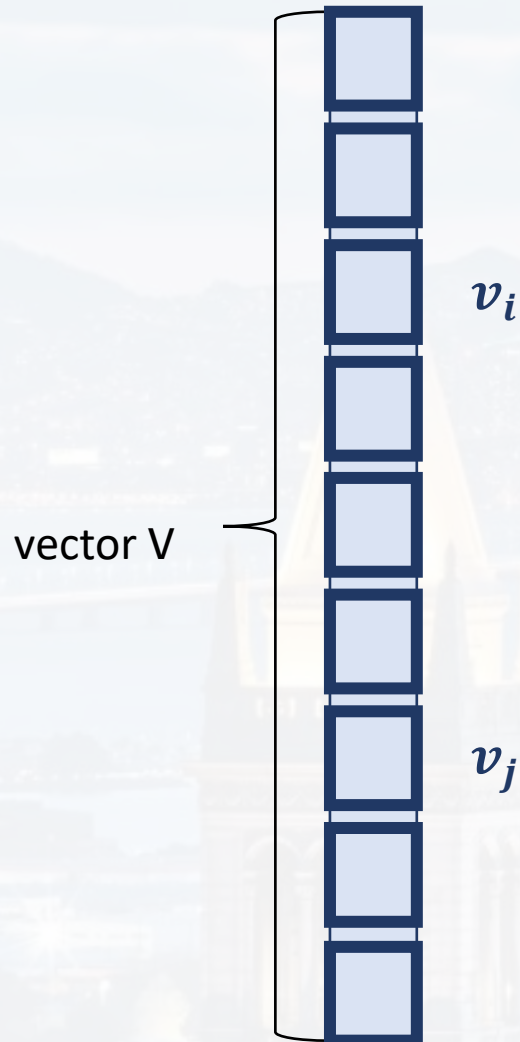
calculating the distance between each element

$$d(v_i, v_j) = (v_i - v_j)^2$$



for loops are simple, but slow → use lin algebra or numpy commands

calculating distances d



calculating the distance between each element

$$d(\mathbf{v}_i, \mathbf{v}_j) = (\mathbf{v}_i - \mathbf{v}_j)^2$$

efficiency:

vector of length N → N*N operations

we know that the diagonal $d(\mathbf{v}_i, \mathbf{v}_i) = 0$ → N fewer operations

only half the operations are necessary: $d(\mathbf{v}_i, \mathbf{v}_j) = d(\mathbf{v}_j, \mathbf{v}_i)$

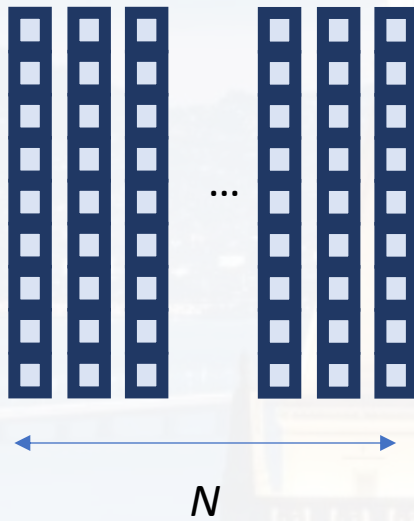
→ instead of N*N operations: only (N-1)*(N-1)/2 needed



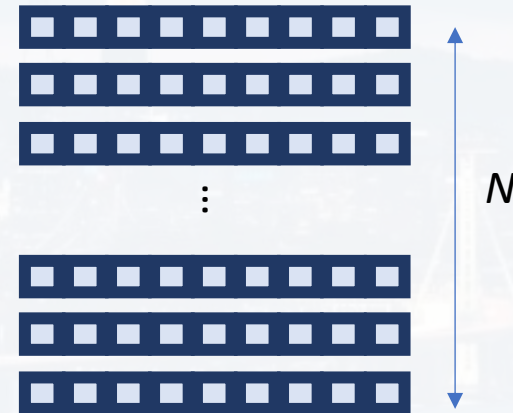
`for` loops are simple, but slow $\rightarrow$ use lin algebra or numpy commands

calculating distances d

```
Rep = np.tile(V, (N,1))
```



```
D = Rep.transpose()
```



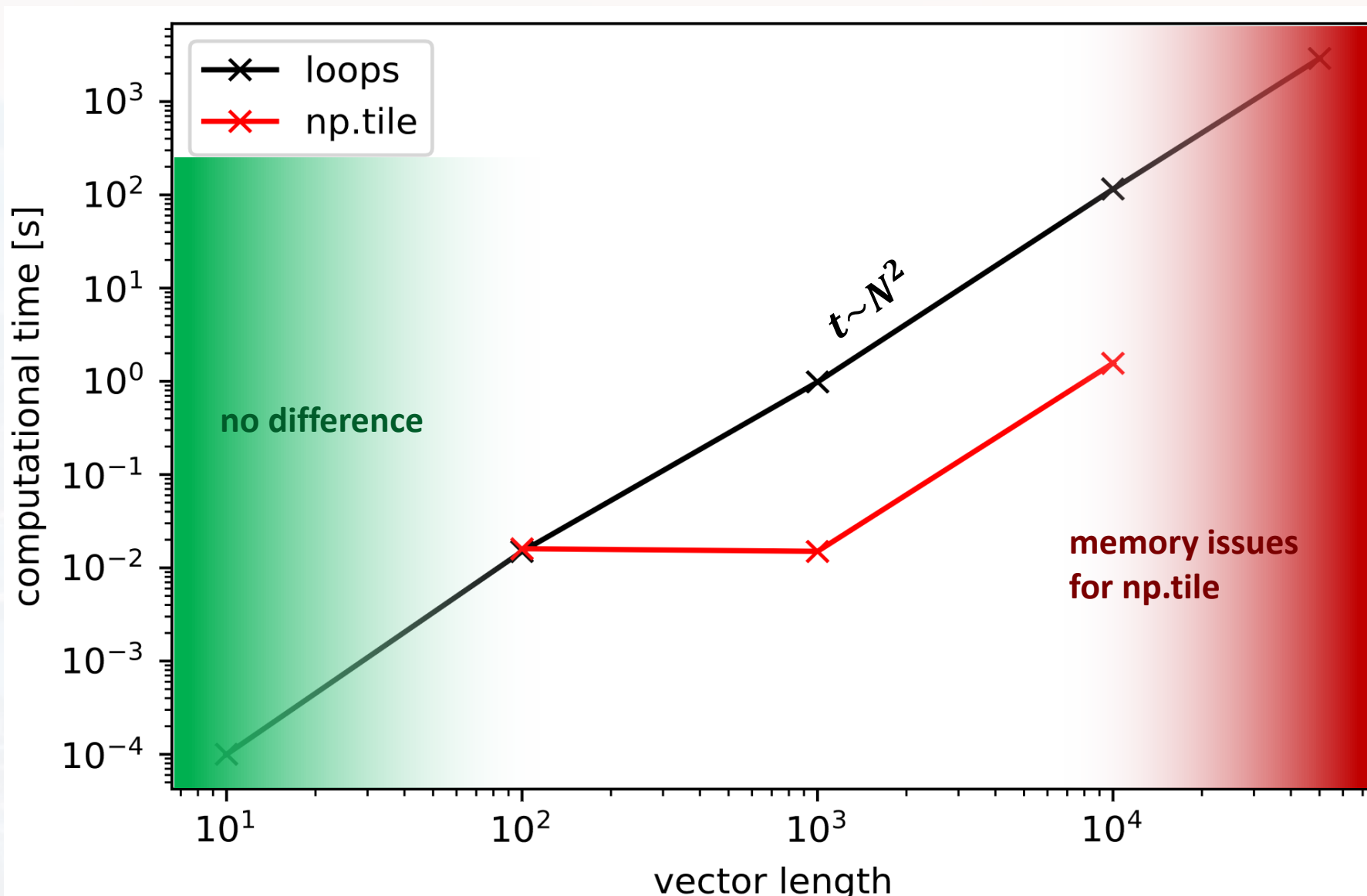
```
D = (Rep - Rep.transpose())**2
```



	loop	efficient loop	np.tile
N = 500	0.11 sec	0.05 sec	0.0003 sec
N = 10,000	35.0 sec	17.8 sec	1.25 sec
N = 50,000			180 sec



see 02_Lecture_Exercise.ipynb





Outline

- Arrays in Python
- Vectors
- Matrices
- Lecture Exercise
- **Solving Linear Systems**



finding the intersection of **two lines**:

$$y_1 = a_1x_1 + c_1$$

$$y_2 = a_2x_2 + c_2$$

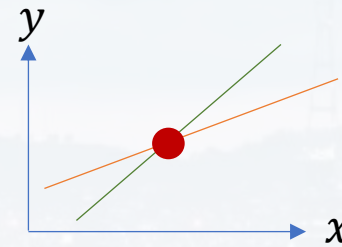
$$x_1 = x_2$$

$$y_1 = y_2$$

$$a_2x + c_2 = a_1x + c_1$$

$$x = \frac{c_2 - c_1}{a_1 - a_2}$$

$$y = a_1 \frac{c_2 - c_1}{a_1 - a_2} + c_1$$



finding the intersection of **three planes**:

$$y_1 = a_{11}x_{11} + a_{12}x_{12} + c_1$$

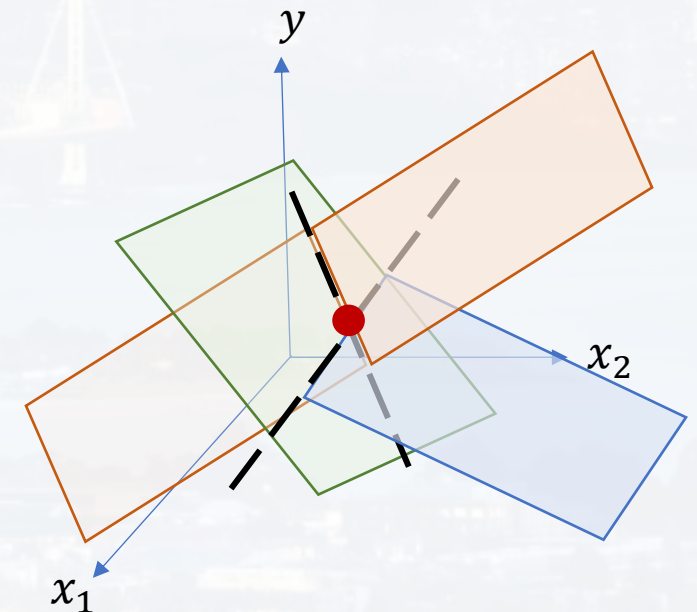
$$y_2 = a_{21}x_{21} + a_{22}x_{22} + c_2$$

$$y_3 = a_{31}x_{31} + a_{32}x_{32} + c_3$$

$$x_{11} = x_{21} = x_{31} = x_1$$

$$x_{12} = x_{22} = x_{32} = x_2$$

$$y_1 = y_2 = y_3 = y$$





more general:

$$x_{11} = x_{21} = x_{31} = x_1$$

$$x_{12} = x_{22} = x_{32} = x_2$$

$$y_1 = y_2 = y_3 = y$$

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \dots + a_{1n}x_n = c_1$$

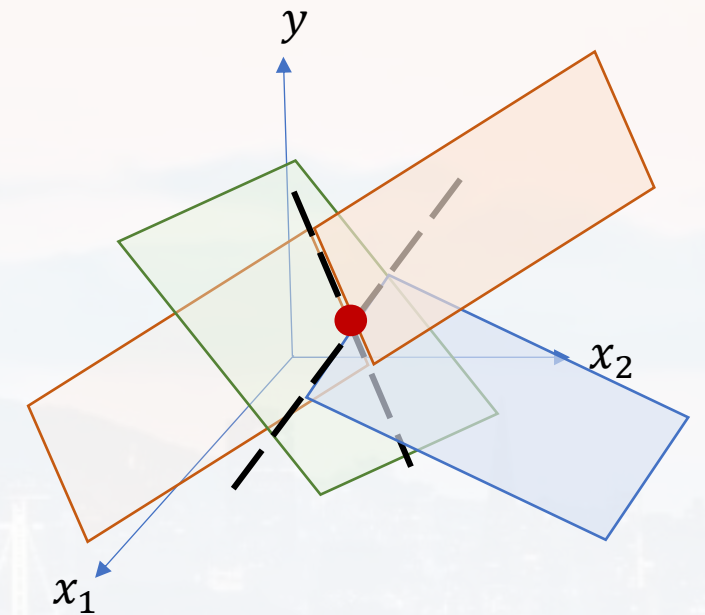
$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \dots + a_{2n}x_n = c_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + \dots + a_{3n}x_n = c_3$$

...

$$a_{m1}x_1 + a_{m2}x_2 + a_{m3}x_3 + \dots + a_{mn}x_n = c_m$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_m \end{bmatrix}$$



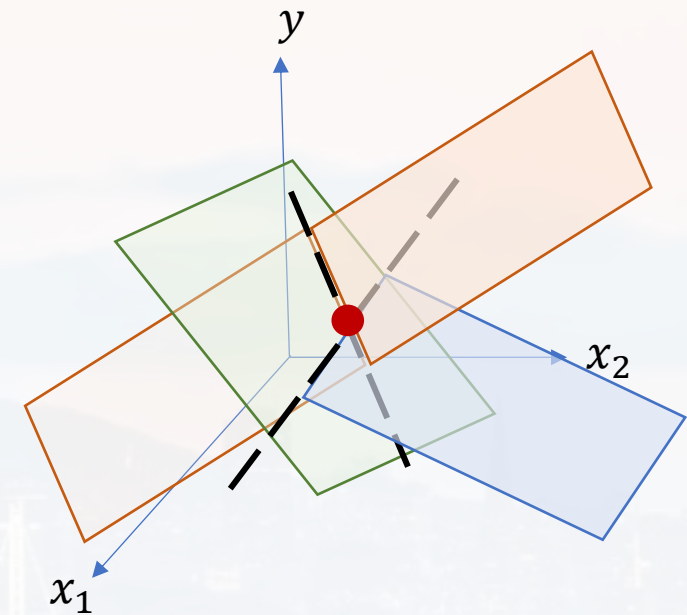


more general:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}}_A \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_m \end{bmatrix}$$

$\vec{x}$ $\vec{c}$

$$A\vec{x} = \vec{c}$$



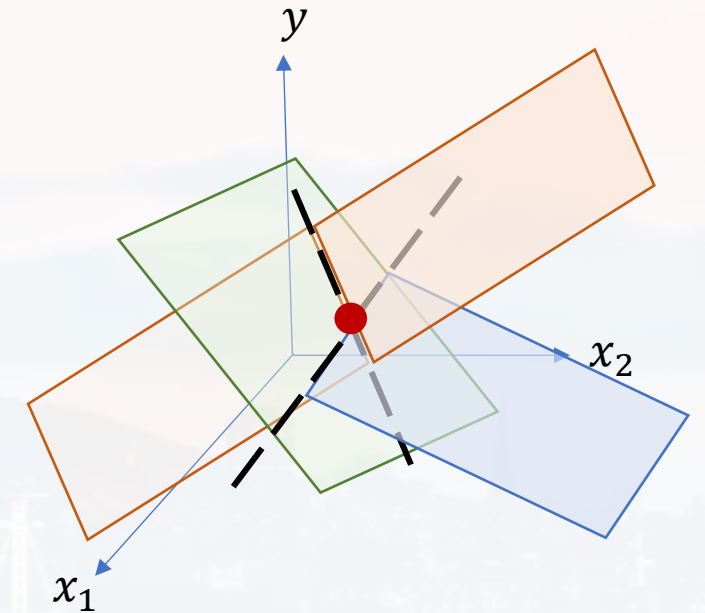


more general:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}}_A \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_m \end{bmatrix}$$

$\vec{x} \quad \vec{c}$

$$A\vec{x} = \vec{c}$$



general set of solutions

for $n = m \rightarrow$ solution is unique: a point

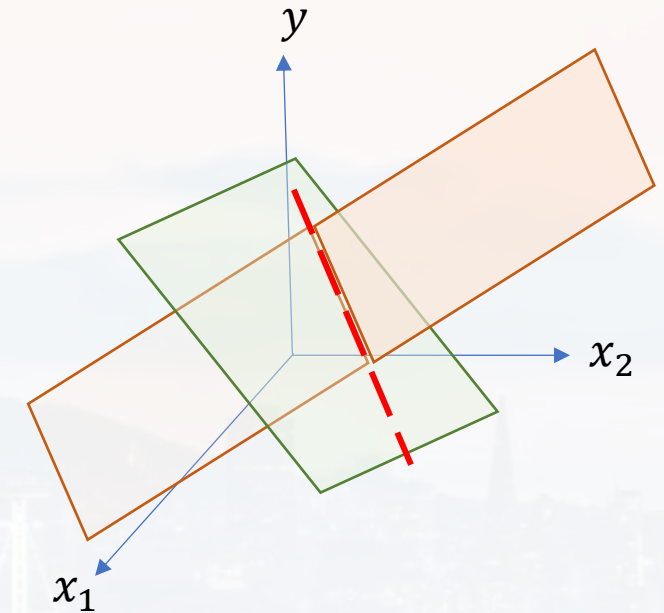


more general:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}}_A \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_m \end{bmatrix}$$

$\vec{x} \quad \vec{c}$

$$A\vec{x} = \vec{c}$$



general set of solutions

for $n = m \rightarrow$ solution is unique: a point

for $n > m$ (more variables than equations)
 $\rightarrow$ solution is not unique: line, hyperplane

for $n < m$ (more equations than variables)
 $\rightarrow$ no solution

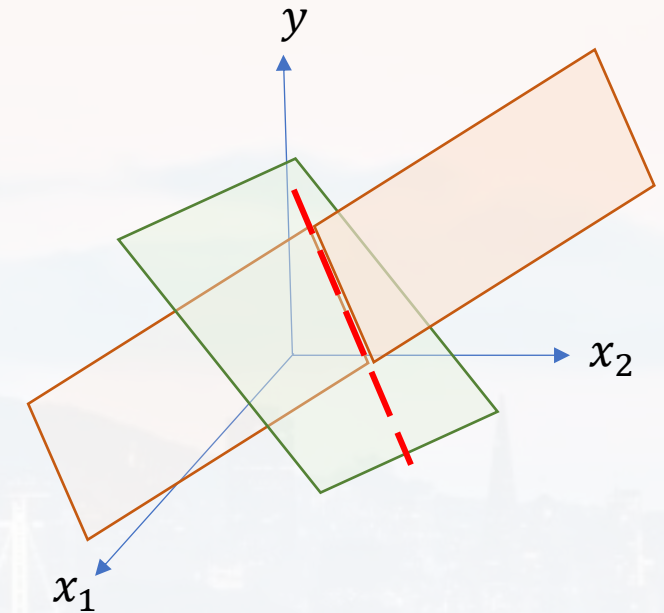


more general:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}}_A \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_m \end{bmatrix}$$

$\vec{x} \quad \vec{c}$

$$A\vec{x} = \vec{c}$$



general set of solutions

for $n = m \rightarrow$ solution is unique: a point

for $n > m$ (more variables than equations)
 $\rightarrow$ solution is not unique: line, hyperplane

for $n < m$ (more equations than variables)
 $\rightarrow$ no solution

exceptions!



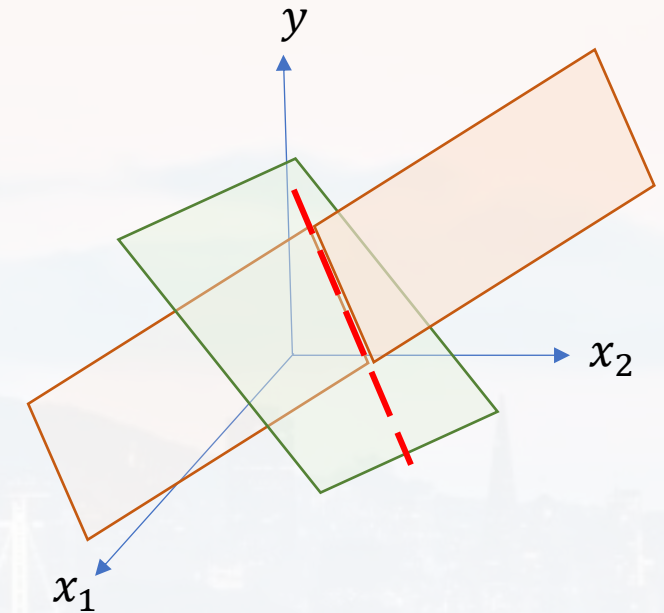


more general:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}}_A \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_m \end{bmatrix}$$

$\vec{x} \quad \vec{c}$

$$A\vec{x} = \vec{c}$$



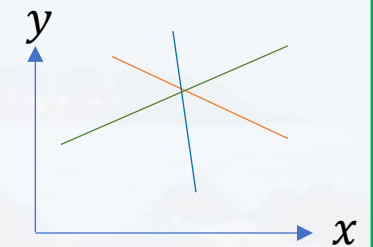
general set of solutions

for $n = m \rightarrow$ solution is unique: a point

for $n > m$ (more variables than equations)
 $\rightarrow$ solution is not unique: line, hyperplane

for $n < m$ (more equations than variables)
 $\rightarrow$ no solution

exceptions!





more general:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \cdots & a_{mn} \end{bmatrix}}_A \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_m \end{bmatrix}$$

$\vec{x} \quad \vec{c}$

$$A\vec{x} = \vec{c} \quad \vec{x} = ?$$

for $n = m$

$$A^{-1}A\vec{x} = A^{-1}\vec{c}$$

$$\vec{x} = A^{-1}\vec{c}$$



solving for $\vec{x}$:

$$\underbrace{\begin{bmatrix} a_{11} & a_{12} & a_{13} & \cdots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \cdots & a_{2n} \\ a_{31} & a_{32} & a_{33} & \cdots & a_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \cdots & a_{nn} \end{bmatrix}}_{A} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ \vdots \\ c_n \end{bmatrix}$$

one possibility: **Gaussian elimination**

idea: it is a system of linear equations, hence we can

- swap rows
- subtract rows from each other
- multiply rows with any non-zero number

goal: turning this into the **identity matrix** / by performing the above operations

→ $\vec{c}$ must be the solution $\vec{x}$

$$\underbrace{\begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix}}_A \underbrace{\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}}_{\vec{x}} = \underbrace{\begin{pmatrix} 8 \\ 11 \\ -3 \end{pmatrix}}_{\vec{c}} \quad \longrightarrow \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$



solving for $\vec{x}$:

$$\begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 11 \\ -3 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

$$\begin{pmatrix} 2x_1 & x_2 & -x_3 \\ -3x_1 & -x_2 & 2x_3 \\ -2x_1 & x_2 & 2x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 11 \\ -3 \end{pmatrix}$$

adding 3/2 of row 1 to row 2

$$\begin{pmatrix} 2x_1 & x_2 & -x_3 \\ 0 & 0.5x_2 & 0.5x_3 \\ -2x_1 & x_2 & 2x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1 \\ -3 \end{pmatrix}$$

adding row 1 to row 3

$$\begin{pmatrix} 2x_1 & x_2 & -x_3 \\ 0 & 0.5x_2 & 0.5x_3 \\ 0 & 2x_2 & x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1 \\ 5 \end{pmatrix}$$

subtracting 4x row 2 from row 3

$$\begin{pmatrix} 2x_1 & x_2 & -x_3 \\ 0 & 0.5x_2 & 0.5x_3 \\ 0 & 0 & -x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1 \\ 1 \end{pmatrix}$$

lower left looks already like from an identity matrix
→ doing the same for the upper right now



solving for $\vec{x}$:

$$\begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 11 \\ -3 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

$$\begin{pmatrix} 2x_1 & x_2 & -x_3 \\ 0 & 0.5x_2 & 0.5x_3 \\ 0 & 0 & -x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 1 \\ 1 \end{pmatrix}$$

subtracting row 3 from row 1

$$\begin{pmatrix} 2x_1 & x_2 & 0 \\ 0 & 0.5x_2 & 0.5x_3 \\ 0 & 0 & -x_3 \end{pmatrix} = \begin{pmatrix} 7 \\ 1 \\ 1 \end{pmatrix}$$

subtracting 2 x row 2 from row 1

$$\begin{pmatrix} 2x_1 & 0 & -x_3 \\ 0 & 0.5x_2 & 0.5x_3 \\ 0 & 0 & -x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 1 \\ 1 \end{pmatrix}$$

subtracting row 3 from row 1

$$\begin{pmatrix} 2x_1 & 0 & 0 \\ 0 & 0.5x_2 & 0.5x_3 \\ 0 & 0 & -x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix}$$



solving for $\vec{x}$:

$$\begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 8 \\ 11 \\ -3 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$$

$$\begin{pmatrix} 2x_1 & 0 & 0 \\ 0 & 0.5x_2 & 0.5x_3 \\ 0 & 0 & -x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix}$$

adding 0.5 row 3 to row 2

$$\begin{pmatrix} 2x_1 & 0 & 0 \\ 0 & 0.5x_2 & 0 \\ 0 & 0 & -x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ 1.5 \\ 1 \end{pmatrix}$$

dividing row 1 by 2
multiplying row 2 by 2
multiplying row 3 by -1

$$\begin{pmatrix} x_1 & 0 & 0 \\ 0 & x_2 & 0 \\ 0 & 0 & x_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 3 \\ -1 \end{pmatrix}$$



With the same trick, we can calculate the inverse of A , A^{-1}

$$AA^{-1} = I$$

therefore $AI = \begin{pmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$

↑
multiplying with A^{-1}

$$AA^{-1}IA^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \bar{a}_{11} & \bar{a}_{12} & \bar{a}_{13} \\ \bar{a}_{21} & \bar{a}_{22} & \bar{a}_{23} \\ \bar{a}_{31} & \bar{a}_{23} & \bar{a}_{33} \end{pmatrix}$$

goal: turning this into the structure below, by performing the operations from a few slides ago

→ will return A^{-1}



solving for x :

$$A\vec{x} = \vec{c}$$

→ need to calculate A^{-1}

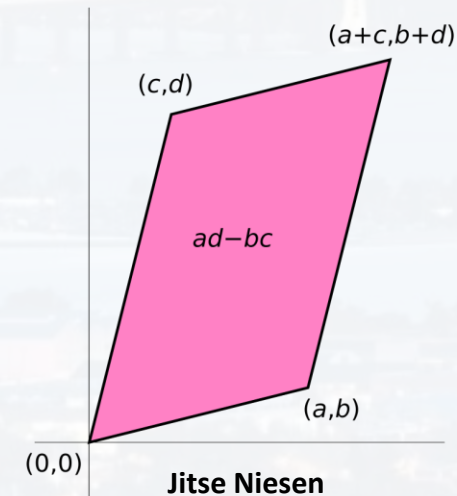
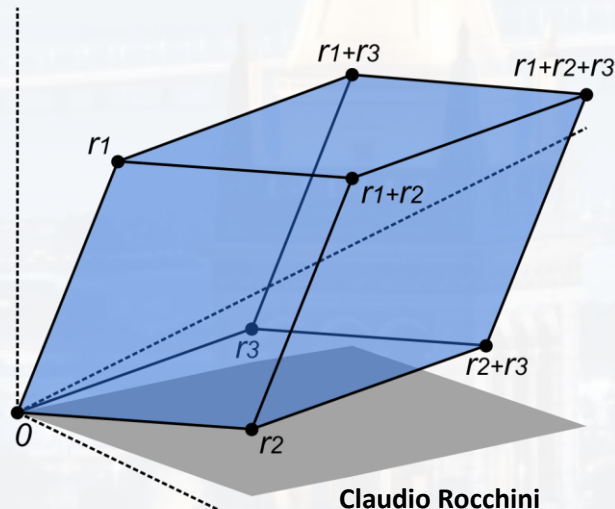
→ need to calculate a quantity called

determinant of A , $\det(A)$

$$A^{-1} \sim \frac{1}{\det(A)}$$

- if $\det(A) = 0 \rightarrow$ no solution

- $|\det(A)|$: volume spanned by the vectors in A





solving for x :

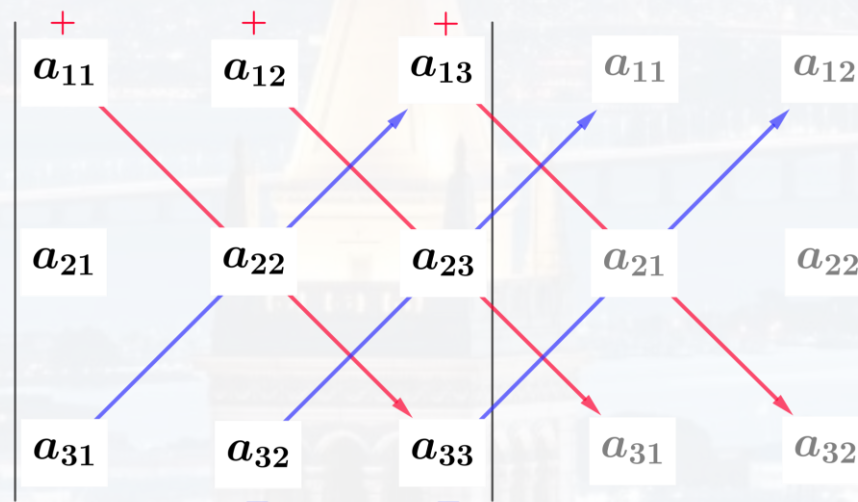
$$A\vec{x} = \vec{c}$$

→ need to calculate A^{-1}

→ need to calculate a quantity called

determinant of A , $\det(A)$

$$A^{-1} \sim \frac{1}{\det(A)}$$



Kmhkmh

$$\det(A) = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{31}a_{22}a_{13} - a_{32}a_{23}a_{11} - a_{33}a_{21}a_{12}$$



solving for x:

$$A\vec{x} = \vec{c}$$

→ need to calculate A^{-1}

→ need to calculate a quantity called

determinant of A , $\det(A)$

$$A^{-1} \sim \frac{1}{\det(A)}$$

N x N matrix:

$$\det(A) = \sum_{i_1, i_2, \dots, i_n} \varepsilon_{i_1 \dots i_n} a_{1, i_1} \dots a_{n, i_n}$$

where

$$\varepsilon_{i_1 \dots i_n} = \prod_{1 \leq \mu < \vartheta \leq n} \text{sgn}(i_\vartheta - i_\mu)$$

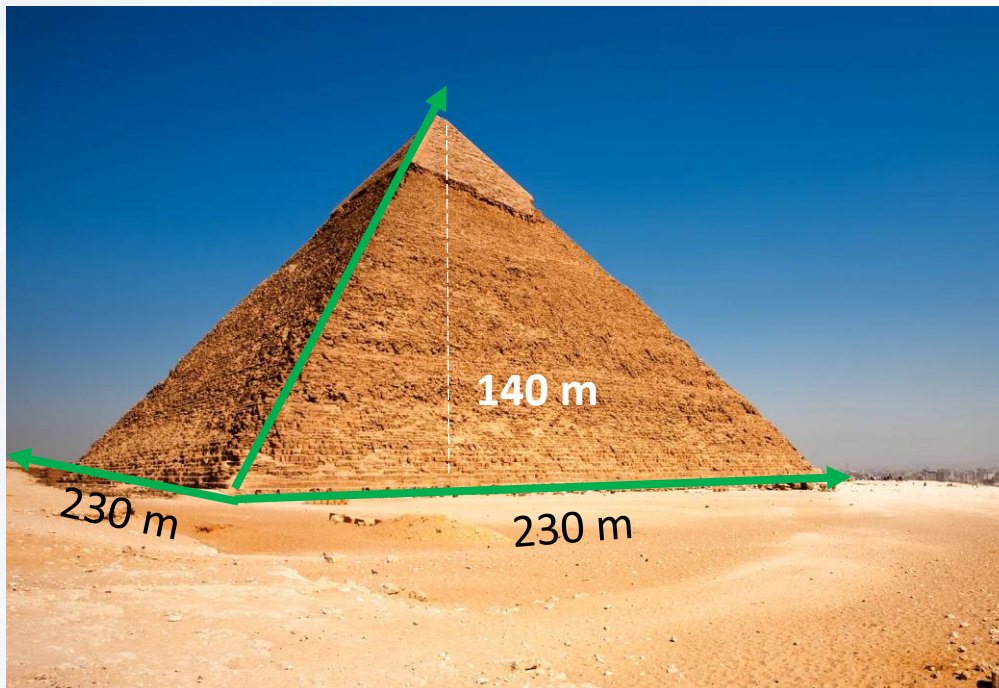
(Levi-Civita symbol)

changing indices
does not change $|\det(A)|$



determinant of A , $\det(A)$

$$A\vec{x} = \vec{c}$$



$$\varepsilon_{i_1 \dots i_n} = \prod_{1 \leq \mu < \nu \leq n} \text{sgn}(i_\nu - i_\mu)$$

$$V = \left| \det \begin{pmatrix} 230 & 0 & 115 \\ 0 & 230 & 115 \\ 0 & 0 & 140 \end{pmatrix} \right| \frac{1}{3} = \frac{230 * 230 * 140 + 0 + 0 - 0 - 0 - 0}{3} = 2,468,666 \text{ m}^3$$



determinant of A, $\det(A)$

$$\varepsilon_{i_1 \dots i_n} = \prod_{1 \leq \mu < \nu \leq n} \text{sgn}(i_\nu - i_\mu)$$



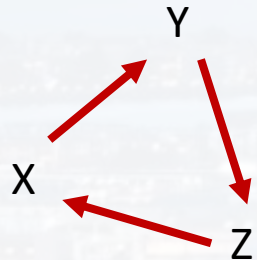
$$V = \left| \det \begin{pmatrix} 230 & 0 & 115 \\ 0 & 230 & 115 \\ 0 & 0 & 140 \end{pmatrix} \right| \frac{1}{3} = \frac{230 * 230 * 140 + 0 + 0 - 0 - 0 - 0}{3} = 2,468,666 \text{ m}^3$$

volume does not
depend on **where**
I put my **coord**
origin...

$$V = \left| \det \begin{pmatrix} 0 & 230 & 115 \\ 230 & 0 & 115 \\ 0 & 0 & 140 \end{pmatrix} \right| \frac{1}{3} = \left| \frac{0 + 0 + 0 - 140 * 230 * 230 - 0 - 0}{3} \right| = 2,468,666 \text{ m}^3$$

...or how I **turn**
the object!

$$V = \left| \det \begin{pmatrix} 115 & 230 & 0 \\ 115 & 0 & 230 \\ 140 & 0 & 0 \end{pmatrix} \right| \frac{1}{3} = \left| \frac{0 + 230 * 230 * 140 + 0 - 0 - 0 - 0}{3} \right| = 2,468,666 \text{ m}^3$$



$$V = \left| \det \begin{pmatrix} 140 & 0 & 0 \\ 115 & 230 & 0 \\ 115 & 0 & 230 \end{pmatrix} \right| \frac{1}{3} = \left| \frac{140 * 230 * 230 + 0 + 0 - 0 - 0 - 0}{3} \right| = 2,468,666 \text{ m}^3$$



$$A\vec{x} = \vec{c}$$

note: - more columns than rows in A

- too many variables, not solvable
- A^{-1} doesn't exist

- fewer columns than rows in A

- too many equations, solution not unique
- A^{-1} doesn't exist

- A^{-1} only **exists in principle if A is a square matrix!**

- even then the system of equations might be **singular** or **degenerate**
→ A^{-1} doesn't exist, $\det(A) = 0$



Thank you for your attention!

